

AI 协作反思日志 — Phase 2 体系结构设计

团队名称：哪一队
日期：2026-04-29
文档版本：V1.0

1. 本阶段 AI 使用清单

任务	AI工具	Prompt摘要	AI输出质量 (1-5)	人工修改幅度(%)
架构风格咨询	DeepSeek	"请推荐一个校园互助服务平台的架构"	2	90%
架构风格咨询	DeepSeek	"4人团队10周Java+Vue，单校3000用户，推荐架构"	5	20%
架构对比分析	DeepSeek	"对比分层/MVC/微服务/事件驱动6个维度"	4	30%
架构师角色咨询	KimiAI	"扮演10年架构师，设计高校信息化项目"	3	60%
模块划分辅助	KimiAI	"基于分层架构划分校园互助平台模块"	4	40%
技术选型咨询	DeepSeek	"Spring Boot项目ORM选MyBatis-Plus还是JPA"	3	50%
ADR初稿生成	DeepSeek&KimiAI	"生成采用分层架构的ADR"	3	70%

2. AI 助力分析（Q1）

本阶段 AI 最大的贡献是：**快速搭建架构设计框架，提供多视角参考**

- 成员B的约束型Prompt直接给出了可落地的10周里程碑，节省了大量调研时间
- 成员C的对比分析帮助团队理解了不同架构的优劣，缩小了决策范围
- AI生成的架构图框架（虽然过度设计）为人工裁剪提供了基础素材

预计节省时间：6H

不同 Prompt 策略下 AI 回答质量的差异：

策略	质量	原因
直接提问	极低	默认推荐流行技术（微服务+Go），完全忽略团队约束
提供详细约束	最高	严格基于条件输出，主动排除不可行方案，给出里程碑
对比多种方案	较高	结构化清晰，但存在概念混淆（MVC vs 分层架构并列）
扮演角色	较高但过度	输出最全面，但"炫技"倾向严重，推荐大量企业级组件

结论：越具体的Prompt，AI输出越务实；角色扮演反而容易过度设计。（然而结合记录人自己的亲身AI使用体验，找到正确的角色扮演方式，或许能进一步提升回答质量）

3. AI 误导分析（Q2）

AI 在架构建议中的典型问题：

- **问题1：默认推荐流行技术，忽略实际约束**
 - 如何发现：成员A直接提问时，AI推荐Go+微服务+Nacos+PostGIS，与团队Java背景、10周工期、3000并发完全脱节
 - 如何修正：补充详细约束后重新提问，人工排除明显不匹配的组件
- **问题2：回避明确决策，给出"两者都可以"的安全回答**
 - 如何发现：咨询ORM选型时，AI说"MyBatis-Plus和JPA都是主流，都可以"，未考虑团队"需大幅依靠AI辅助"的特殊约束
 - 如何修正：人工明确选择MyBatis-Plus，补充"AI生成SQL可控性"这一关键维度
- **问题3：过度设计，不擅长做减法**
 - 如何发现：扮演架构师时，AI推荐RabbitMQ、XXL-JOB、MinIO、Elasticsearch、SonarQube等，对10周学生项目明显过重
 - 如何修正：人工裁剪组件，用Redis+定时任务替代RabbitMQ，本地存储替代MinIO，放弃Elasticsearch
- **问题4：声称"学习成本低"但实际不准确**
 - 如何发现：AI推荐RabbitMQ时称"团队学习成本低"，但实际需理解交换机、队列绑定、死信队列等概念
 - 如何修正：基于团队实际能力判断，10周内无法掌握MQ的运维和排错

4. Prompt 改进计划（Q3）

上阶段（P1）提出的改进措施，验证结果：

P1改进措施	验证结果
细化提示词，给出明确细节要求	✅ 有效：成员B的详细约束Prompt输出质量最高
给出严格的约束条件	✅ 有效：AI主动排除了微服务和IM系统
要求AI输出异常流和可测试标准	★ 部分有效：架构阶段难以直接套用，需转化为"排除不可行方案"

本阶段新的改进措施：

1. **明确告知AI"不要做X"**：在有较为明确的想法和约束的条件下，在Prompt中加入负面约束（如"不要推荐微服务"、"不要推荐Go语言"），比只说"推荐合适的"更有效
2. **要求AI给出"不做XX的理由"**：强迫AI做减法，而非堆砌组件
3. **多轮追问裁剪**：第一轮获取完整方案，第二轮专门追问"哪些组件可以去掉"，第三轮确认最小可行集合
4. **角色扮演需谨慎**："架构师"角色容易触发AI的"炫技"模式，学生项目建议用类似"资深但务实的工程师"角色（待探索）

5. 本阶段的核心工程判断

决策内容：**不引入RabbitMQ，用Redis + Spring定时任务实现订单超时取消**

为什么 AI 无法做这个决策：

AI可以列出RabbitMQ和定时任务的优缺点，甚至可以写两者的实现代码。但AI无法理解"10周"对我们意味着什么——不是"10周内学会RabbitMQ"，而是"10周内要交付一个能跑起来的系统，后面还有P3-P7的其他阶段"。

AI不清楚团队成员的技术掌握实际情况，所推荐的"学习成本低"是基于理想化的学习者，但不一定贴合我们的学习能力和时间安排。

这个决策需要代入**具体的团队能力、时间压力、运维风险**来判断，这些是AI无法获取的上下文。AI只能告诉我们"RabbitMQ很强大"，但是否必需需要我们自己做判断。

记录人：LXY

审核状态：综合全部团队成员AI使用情况，已经由人工审查